

Grandmaster level in StarCraft II using multi-agent reinforcement learning

refers to tools, not human being simulators

<https://doi.org/10.1038/s41586-019-1724-z>

Received: 30 August 2019

Accepted: 10 October 2019

Published online: 30 October 2019

Oriol Vinyals^{1,3*}, Igor Babuschkin^{1,3}, Wojciech M. Czarnecki^{1,3}, Michaël Mathieu^{1,3}, Andrew Dudzik^{1,3}, Junyoung Chung^{1,3}, David H. Choi^{1,3}, Richard Powell^{1,3}, Timo Ewalds^{1,3}, Petko Georgiev^{1,3}, Junhyuk Oh^{1,3}, Dan Horgan^{1,3}, Manuel Kroiss^{1,3}, Ivo Danihelka^{1,3}, Aja Huang^{1,3}, Laurent Sifre^{1,3}, Trevor Cai^{1,3}, John P. Agapiou^{1,3}, Max Jaderberg¹, Alexander S. Vezhnevets¹, Rémi Leblond¹, Tobias Pohlen¹, Valentin Dalibard¹, David Budden¹, Yury Sulsky¹, James Molloy¹, Tom L. Paine¹, Caglar Gulcehre¹, Ziyu Wang¹, Tobias Pfaff¹, Yuhuai Wu¹, Roman Ring¹, Dani Yogatama¹, Dario Wünsch², Katrina McKinney¹, Oliver Smith¹, Tom Schaul¹, Timothy Lillicrap¹, Koray Kavukcuoglu¹, Demis Hassabis¹, Chris Apps^{1,3} & David Silver^{1,3*}

Many real-world applications require artificial agents to compete and coordinate with other agents in complex environments. As a stepping stone to this goal, the domain of StarCraft has emerged as an important challenge for artificial intelligence research, owing to its iconic and enduring status among the most difficult professional esports and its relevance to the real world in terms of its raw complexity and multi-agent challenges. Over the course of a decade and numerous competitions^{1–3}, the strongest agents have simplified important aspects of the game, utilized superhuman capabilities, or employed hand-crafted sub-systems⁴. Despite these advantages, no previous agent has come close to matching the overall skill of top StarCraft players. We chose to address the challenge of StarCraft using general-purpose learning methods that are in principle applicable to other complex domains: a multi-agent reinforcement learning algorithm that uses data from both human and agent games within a diverse league of continually adapting strategies and counter-strategies, each represented by deep neural networks^{5,6}. We evaluated our agent, AlphaStar, in the full game of StarCraft II, through a series of online games against human players. AlphaStar was rated at Grandmaster level for all three StarCraft races and above 99.8% of officially ranked human players.

StarCraft is a real-time strategy game in which players balance high-level economic decisions with individual control of hundreds of units. This domain raises important game-theoretic challenges: it features a vast space of cyclic, non-transitive strategies and counter-strategies; discovering novel strategies is intractable with naive self-play exploration methods; and those strategies may not be effective when deployed in real-world play with humans. Furthermore, StarCraft has a combinatorial action space, a planning horizon that extends over thousands of real-time decisions, and imperfect information⁷.

Each game consists of tens of thousands of time-steps and thousands of actions, selected in real-time throughout approximately ten minutes of gameplay. At each step t , our agent AlphaStar receives an observation o_t that includes a list of all observable units and their attributes. This information is imperfect; the game includes only opponent units seen by the player's own units, and excludes some opponent unit attributes outside the camera view.

Each action a_t is highly structured: it selects what action type, out of several hundred (for example, move or build worker); who to issue that action to, for any subset of the agent's units; where to target, among locations on the map or units within the camera view; and when to observe and act next (Fig. 1a). This representation of actions results in approximately 10^{26} possible choices at each step. Similar to human players, a special action is available to move the camera view, so as to gather more information.

Humans play StarCraft under physical constraints that limit their reaction time and the rate of their actions. The game was designed with those limitations in mind, and removing those constraints changes the nature of the game. We therefore chose to impose constraints upon AlphaStar: it suffers from delays due to network latency and computation time; and its actions per minute (APM) are limited, with peak statistics substantially lower than those of humans (Figs. 2c, 3g for performance analysis). AlphaStar's play with this interface and these

¹DeepMind, London, UK. ²Team Liquid, Utrecht, Netherlands. ³These authors contributed equally: Oriol Vinyals, Igor Babuschkin, Wojciech M. Czarnecki, Michaël Mathieu, Andrew Dudzik, Junyoung Chung, David H. Choi, Richard Powell, Timo Ewalds, Petko Georgiev, Junhyuk Oh, Dan Horgan, Manuel Kroiss, Ivo Danihelka, Aja Huang, Laurent Sifre, Trevor Cai, John P. Agapiou, Chris Apps, David Silver. *e-mail: vinyals@google.com; davidsilver@google.com

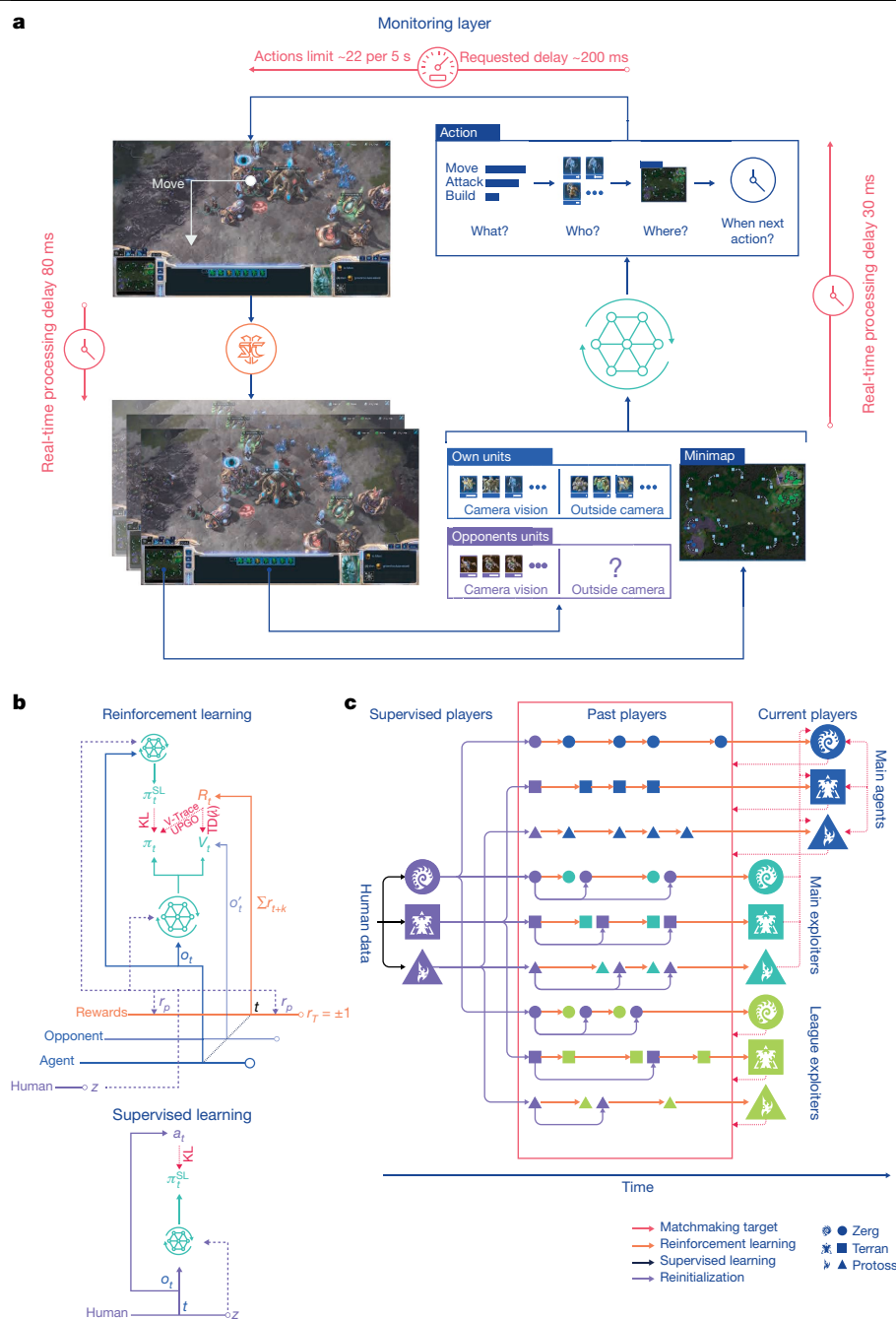


Fig. 1 | Training setup. **a**, AlphaStar observes the game through an overview map and list of units. To act, the agent outputs what action type to issue (for example, build), who it is applied to, where it targets, and when the next action will be issued. Actions are sent to the game through a monitoring layer that limits action rate. AlphaStar contends with delays from network latency and processing time. **b**, AlphaStar is trained via both supervised learning and reinforcement learning. In supervised learning (bottom), the parameters are updated to optimize Kullback–Leibler (KL) divergence between its output and human actions sampled from a collection of replays. In reinforcement learning (top), human data are used to sample the statistic z , and agent experience is collected to update the policy and value outputs via reinforcement learning (TD(λ), V-trace, UPGO) combined with a KL loss towards the supervised agent. **c**, Three pools of agents, each initialized by supervised learning, were subsequently trained with reinforcement learning. As they train, these agents intermittently add copies of themselves—‘players’ that are frozen at a specific point—to the league. The main agents train against all of these past players, as well as themselves. The league exploiters train against all past players. The main exploiters train against the main agents. Main exploiters and league exploiters can be reset to the supervised agent when they add a player to the league. Images from StarCraft reproduced with permission from Blizzard Entertainment.

constraints was approved by a professional player (see ‘Professional player statement’ in Methods).

Learning algorithm

To address the complexity and game-theoretic challenges of StarCraft, AlphaStar uses a combination of new and existing general-purpose techniques for neural network architectures, imitation learning, reinforcement learning, and multi-agent learning. Further details about these techniques are given in the Methods.

Central to AlphaStar is a policy $\pi_{\theta}(a_t | s_t, z) = \mathbb{P}[a_t | s_t, z]$, represented by a neural network with parameters θ that receives all observations $s_t = (o_{1:t}, a_{1:t-1})$ from the start of the game as inputs, and selects actions as outputs. The policy is also conditioned on a statistic z that summarizes a strategy sampled from human data (for example, a build order).

Our agent architecture consists of general-purpose neural network components that handle StarCraft’s raw complexity. Observations of

player and opponent units are processed using a self-attention mechanism⁸. To integrate spatial and non-spatial information, we introduce scatter connections. To deal with partial observability, the temporal sequence of observations is processed by a deep long short-term memory (LSTM) system⁹. To manage the structured, combinatorial action space, the agent uses an auto-regressive policy^{7,10,11} and recurrent pointer network¹². Extended Data Fig. 3 summarizes the architecture and Fig. 3f shows an ablation of each component.

Agent parameters were initially trained by supervised learning. Games were sampled from a publicly available dataset of anonymized human replays. The policy was then trained to predict each action a_t , conditioned either solely on s_t , or also on z . This results in a diverse set of strategies that reflects the modes of human play.

The agent parameters were subsequently trained by a reinforcement learning algorithm that is designed to maximize the win rate (that is, compute a best response) against a mixture of opponents. The choice of opponent is determined by a multi-agent procedure, described

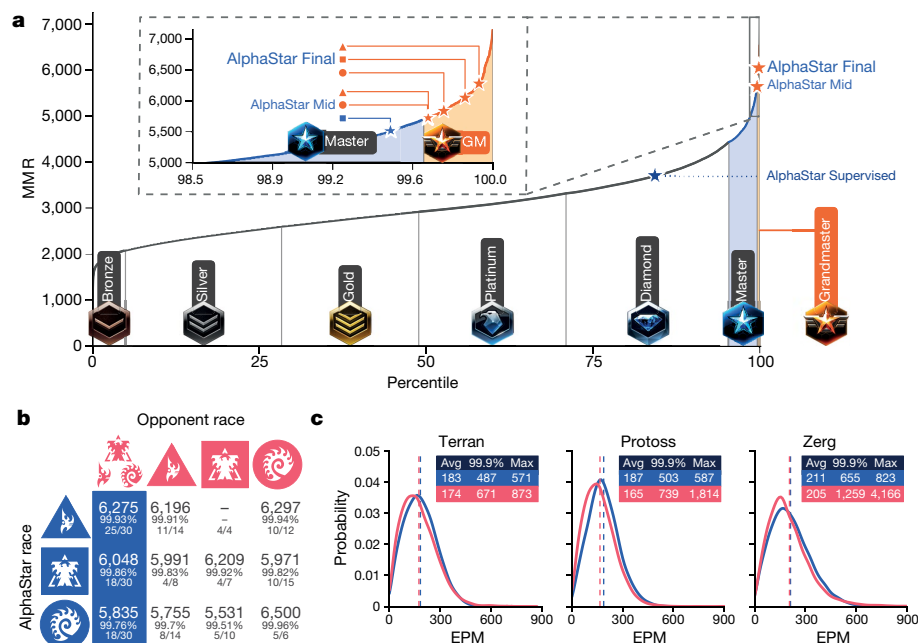


Fig. 2 | Results. **a**, On Battle.net, StarCraft II players are divided into seven leagues, from Bronze to Grandmaster, according to their ratings (MMR). We played three variants of AlphaStar on Battle.net: AlphaStar Supervised, AlphaStar Mid, and AlphaStar Final. The supervised agent was rated in the top 16% of human players, the midpoint agent within the top 0.5%, and the final agent, on average, within the top 0.15%, achieving a Grandmaster level rating for all three races. **b**, MMR ratings of AlphaStar Final per race (from top to

bottom: Protoss, Terran, Zerg) versus opponents encountered on Battle.net (from left to right: all races combined, Protoss, Terran, Zerg). Note that per-race data are limited; AlphaStar won all Protoss versus Terran games. **c**, Distribution of effective actions per minute (EPM) as reported by StarCraft II for both AlphaStar Final (blue) and human players (red). Dashed lines show mean values. Icons reproduced with permission from Blizzard Entertainment.

below. AlphaStar's reinforcement learning algorithm is based on a policy gradient algorithm similar to advantage actor-critic¹³. Updates were applied asynchronously¹⁴ on replayed experiences¹⁵. This requires an approach known as off-policy learning⁵, that is, updating the current policy from experience generated by a previous policy. Our solution is motivated by the observation that, in large action spaces, the current and previous policies are highly unlikely to match over many steps. We therefore use a combination of techniques that can learn effectively despite the mismatch: temporal difference learning (TD(λ))¹⁶, clipped importance sampling (V-trace)¹⁴, and a new self-imitation¹⁷ algorithm (UPGO) that moves the policy towards trajectories with better-than-average reward. To reduce variance, during training only, the value function is estimated using information from both the player's and the opponent's perspectives. Figure 3i, k analyses the relative importance of these components.

One of the main challenges in StarCraft is to discover novel strategies. Consider a policy that has learned to build and utilize the micro-tactics of ground units. Any deviation that builds and naively uses air units will reduce performance. It is highly improbable that naive exploration will execute a precise sequence of instructions, over thousands of steps, that constructs air units and effectively utilizes their micro-tactics. To address this issue, and to encourage robust behaviour against likely human play, we utilize human data. Each agent is initialized to the parameters of the supervised learning agent. Subsequently, during reinforcement learning, we either condition the agent on a statistic z , in which case agents receive a reward for following the strategy corresponding to z , or train the agent unconditionally, in which case the agent is free to choose its own strategy. Agents also receive a penalty whenever their action probabilities differ from the supervised policy. This human exploration ensures that a wide variety of relevant modes of play continue to be explored throughout training. Figure 3e shows the importance of human data in AlphaStar.

To address the game-theoretic challenges, we introduce league training, an algorithm for multi-agent reinforcement learning (Fig. 1b, c). Self-play algorithms, similar to those used in chess and Go¹⁸, learn rapidly but may chase cycles (for example, where A defeats B, and B defeats C, but A loses to C) indefinitely without making progress¹⁹. Fictitious self-play (FSP)^{20–22} avoids cycles by computing a best response against a uniform mixture of all previous policies; the mixture converges to a Nash equilibrium in two-player zero-sum games²⁰. We extend this approach to compute a best response against a non-uniform mixture of opponents. This league of potential opponents includes a diverse range of agents (Fig. 4d), as well as their policies from both current and previous iterations. At each iteration, each agent plays games against opponents sampled from a mixture policy specific to that agent. The parameters of the agent are updated from the outcomes of those games by the actor-critic reinforcement learning procedure described above.

The league consists of three distinct types of agent, differing primarily in their mechanism for selecting the opponent mixture. First, the main agents utilize a prioritized fictitious self-play (PFSP) mechanism that adapts the mixture probabilities proportionally to the win rate of each opponent against the agent; this provides our agent with more opportunities to overcome the most problematic opponents. With fixed probability, a main agent is selected as an opponent; this recovers the rapid learning of self-play (Fig. 3c). Second, main exploiter agents play only against the current iteration of main agents. Their purpose is to identify potential exploits in the main agents; the main agents are thereby encouraged to address their weaknesses. Third, league exploiter agents use a similar PFSP mechanism to the main agents, but are not targeted by main exploiter agents. Their purpose is to find systemic weaknesses of the entire league. Both main exploiters and league exploiters are periodically reinitialized to encourage more diversity and may rapidly discover specialist strategies that are not necessarily robust against exploitation. Figure 3b analyses the choice of agents within the league.

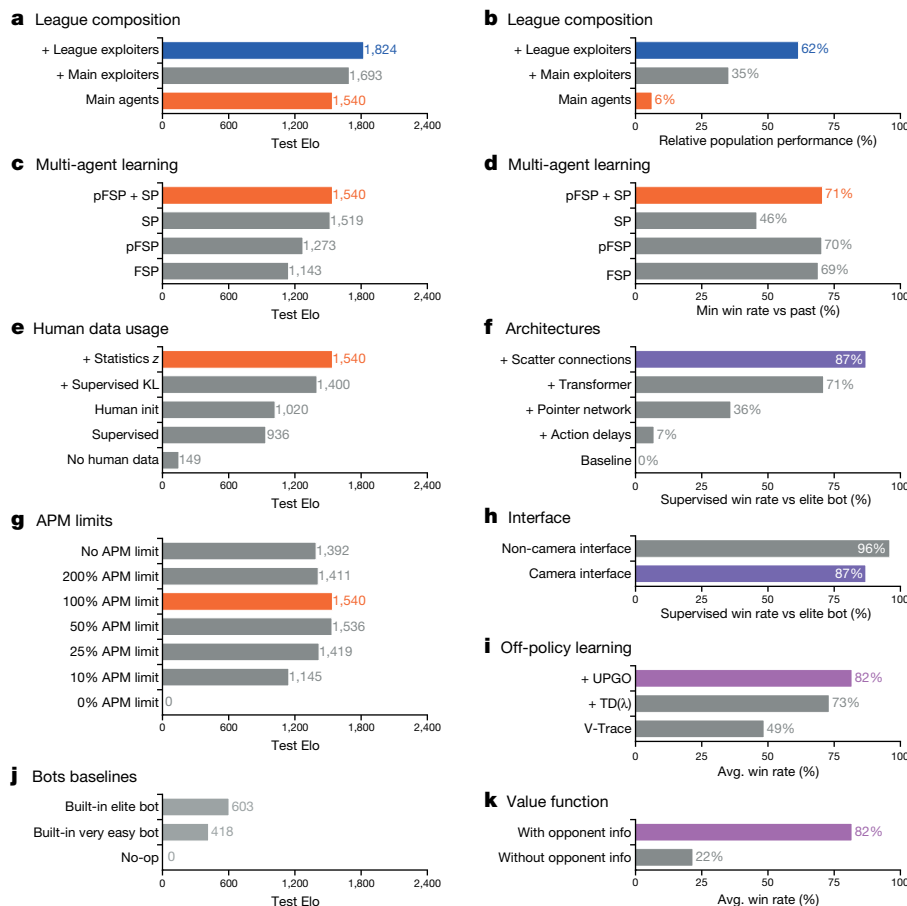


Fig. 3 | Ablations for key components of AlphaStar.

These experiments use a simplified setup: one map (Kairos Junction), one race match-up (Protoss versus Protoss), reinforcement learning and league experiments limited to 10^{10} steps, only main agents, and a 50%–50% mix of self-play and PFSP, unless stated otherwise (see Methods). The first column shows Elo ratings²⁴ against ablation test agents (each rating was estimated with 11,000 full games of StarCraft II). **a, b**, Comparing different league compositions using Elo of the main agents (**a**) and relative population performance of the whole leagues (**b**), which measures exploitability. **c, d**, Comparing different multi-agent learning algorithms using Elo (**c**) and a proxy for forgetting: the minimum win rate against all past versions, averaged over time (**d**). Naive self-play has a high Elo, but is more forgetful. See Extended Data Fig. 5 for more in-depth comparison. **e**, Ablation study of the different mechanisms to use human data. Human init, supervised learning initialization of parameters of the neural network. **g**, APM limits relative to those used in AlphaStar. Reducing APM substantially reduces performance. Unexpectedly, increasing APM also reduces performance, possibly because the agent spends more effort on refining micro-tactics than on learning diverse strategies. **f, h**, Comparison of architectures using the win rate of supervised agents (trained in Protoss versus all) against the built-in elite bot. **j**, Elo scores of StarCraft II built-in bots. Ratings are anchored by a bot that never acts. **i, k**, Reinforcement learning ablations, measured by training a best response against fixed opponents to avoid multi-agent dynamics.

In StarCraft, each player chooses one of three races—Terran, Protoss or Zerg—each with distinct mechanics. We trained the league using three main agents (one for each StarCraft race), three main exploiter agents (one for each race), and six league exploiter agents (two for each race). Each agent was trained using 32 third-generation tensor processing units (TPUs²³) over 44 days. During league training almost 900 distinct players were created.

Empirical evaluation

We evaluated the three main Terran, Protoss and Zerg AlphaStar agents using the unconditional policy on the official online matchmaking system Battle.net. Each agent was assessed at three different snapshots during training: after supervised training only (AlphaStar Supervised), after 27 days of league training (AlphaStar Mid), and after 44 days of league training (AlphaStar Final). AlphaStar Supervised and AlphaStar Mid were evaluated starting from an unranked rating on Battle.net for 30 and 60 games, respectively, for each race; AlphaStar Final was evaluated from AlphaStar Mid's rating for an additional 30 games for each race. The Battle.net matchmaking procedure selected maps and opponents. Matches were played under blind conditions: AlphaStar was not provided with the opponent's identity, and played under an anonymous account. These conditions were selected to estimate AlphaStar's strength under approximately stationary conditions, but do not directly measure its susceptibility to exploitation under repeated play.

AlphaStar Final achieved ratings of 6,275 Match Making Rating (MMR) for Protoss, 6,048 MMR for Terran and 5,835 MMR for Zerg, placing it above 99.8% of ranked human players, and at Grandmaster level for all three races (Fig. 2a, Extended Data Fig. 7 (analysis), Supplementary Data, Replays (game replays)). AlphaStar Supervised reached an average rating of 3,699, which places it above 84%

of human players and shows the effectiveness of supervised learning.

To further analyse AlphaStar we also ran several internal ablations (Fig. 3) and evaluations (Fig. 4). For multi-agent dynamics, we ran a round-robin tournament of all players throughout league training and a second tournament of main agents against held-out validation agents trained to follow specific human strategies. The main agent performance improved steadily across all three races. The performance of the main exploiters actually reduced over time and main agents performed better against the held-out validation agents, both of which suggest that the main agent grew increasingly robust. The league Nash equilibrium over all players at each point in time assigns small probabilities to players from previous iterations, suggesting that the learning algorithm does not cycle or regress. Finally, the unit composition changed throughout league training, which indicates a diverse strategic progression.

Conclusion

AlphaStar is the first agent to achieve Grandmaster level in StarCraft II, and the first to reach the highest league of human players in a widespread professional esports without simplification of the game. Like StarCraft, real-world domains such as personal assistants, self-driving cars, or robotics require real-time decisions, over combinatorial or structured action spaces, given imperfectly observed information. Furthermore, similar to StarCraft, many applications have complex strategy spaces that contain cycles or hard exploration landscapes, and agents may encounter unexpected strategies or complex edge cases when deployed in the real world. The success of AlphaStar in StarCraft II suggests that general-purpose machine learning algorithms may have a substantial effect on complex real-world problems.

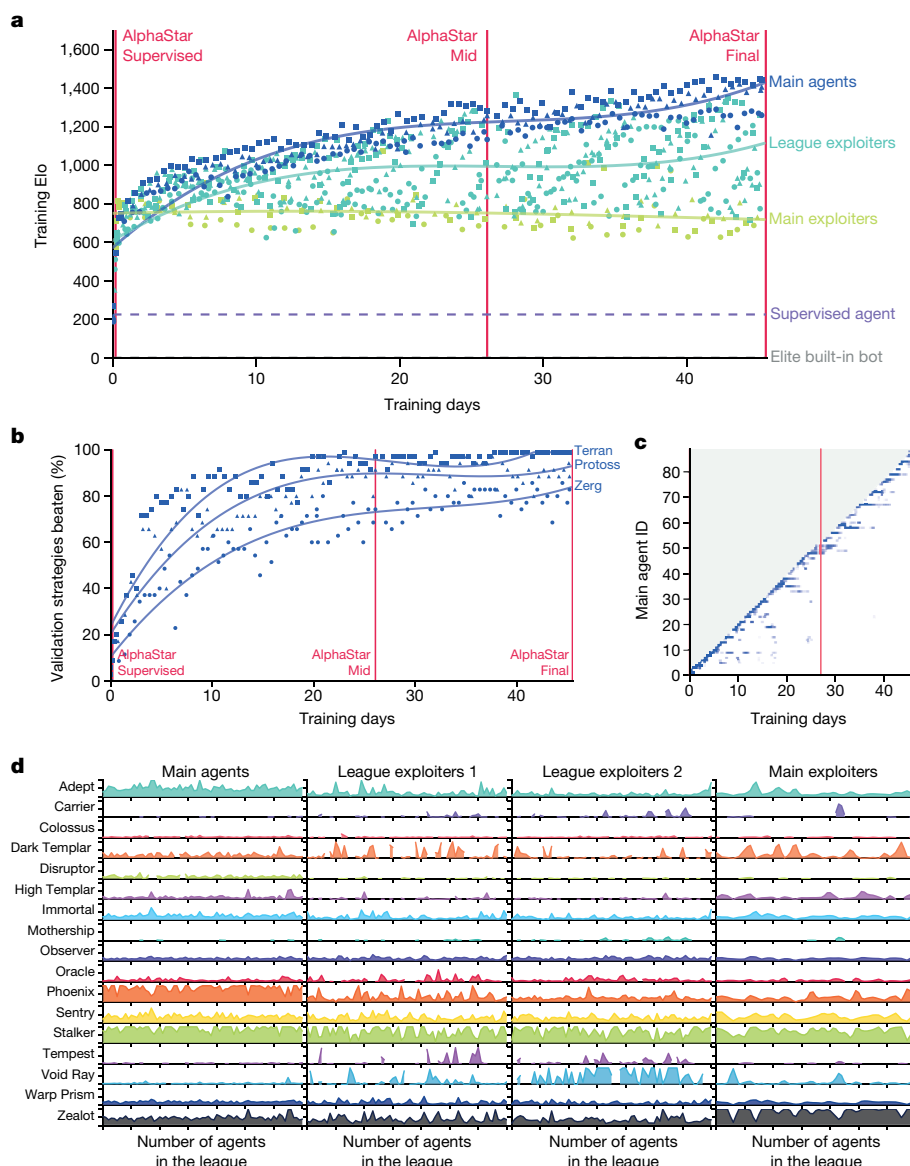


Fig. 4 | AlphaStar training progression. **a**, Training Elo scores of agents in the league during the 44 days of training. Each point represents a past player, evaluated against the entire league and the elite built-in bot (whose Elo is set to 0). **b**, Proportion of validation agents that beat the main agents in more than 80 out of 160 games. This value increased steadily over time, which shows the robustness of league training to unseen strategies. **c**, The Nash distribution (mixture of the least exploitable players) of the players in the league, as training progressed. It puts the most weight on recent players, suggesting that the latest strategies largely dominate earlier ones, without much forgetting or cycling. For example, player 40 was part of the Nash distribution from its creation at day 20 until 5 days later, when it was completely dominated by newer agents. **d**, Average number of each unit built by the Protoss agents over the course of league training, normalized by the most common unit. Unlike the main agents, the exploiters rapidly explore different unit compositions. Worker units have been removed for clarity.

Online content

Any methods, additional references, Nature Research reporting summaries, source data, extended data, supplementary information, acknowledgements, peer review information; details of author contributions and competing interests; and statements of data and code availability are available at <https://doi.org/10.1038/s41586-019-1724-z>.

1. AllIDE StarCraft AI Competition. <https://www.cs.mun.ca/dchurchill/starcraftaicomp/>.
2. Student StarCraft AI Tournament and Ladder. <https://sscaitournament.com/>.
3. Starcraft 2 AI ladder. <https://sc2ai.net/>.
4. Churchill, D., Lin, Z. & Synnaeve, G. An analysis of model-based heuristic search techniques for StarCraft combat scenarios. in *Artificial Intelligence and Interactive Digital Entertainment Conf. (AAAI)*, 2017).
5. Sutton, R. & Barto, A. *Reinforcement Learning: An Introduction* (MIT Press, 1998).
6. LeCun, Y., Bengio, Y. & Hinton, G. Deep learning. *Nature* **521**, 436–444 (2015).
7. Vinyals, O. et al. StarCraft II: a new challenge for reinforcement learning. Preprint at <https://arxiv.org/abs/1708.04782> (2017).
8. Vaswani, A. et al. Attention is all you need. *Adv. Neural Information Process. Syst.* **30**, 5998–6008 (2017).
9. Hochreiter, S. & Schmidhuber, J. Long short-term memory. *Neural Comput.* **9**, 1735–1780 (1997).
10. Mikolov, T., Karafiat, M., Burget, L., Cernocky, J. & Khudanpur, S. Recurrent neural network based language model. *INTERSPEECH-2010* 1045–1048 (2010).
11. Metz, L., Ibarz, J., Jaitly, N. & Davidson, J. Discrete sequential prediction of continuous actions for deep RL. Preprint at <https://arxiv.org/abs/1705.05035v3> (2017).
12. Vinyals, O., Fortunato, M. & Jaitly, N. Pointer networks. *Adv. Neural Information Process. Syst.* **28**, 2692–2700 (2015).

13. Mnih, V. et al. Asynchronous methods for deep reinforcement learning. *Proc. Machine Learning Res.* **48**, 1928–1937 (2016).
14. Espeholt, L. et al. IMPALA: scalable distributed deep-RL with importance weighted actor-learner architectures. *Proc. Machine Learning Res.* **80**, 1407–1416 (2018).
15. Wang, Z. et al. Sample efficient actor-critic with experience replay. Preprint at <https://arxiv.org/abs/1611.01224v2> (2017).
16. Sutton, R. Learning to predict by the method of temporal differences. *Mach. Learn.* **3**, 9–44 (1988).
17. Oh, J., Guo, Y., Singh, S. & Lee, H. Self-Imitation Learning. *Proc. Machine Learning Res.* **80**, 3875–3884 (2018).
18. Silver, D. et al. A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play. *Science* **362**, 1140–1144 (2018).
19. Balduzzi, D. et al. Open-ended learning in symmetric zero-sum games. *Proc. Machine Learning Res.* **97**, 434–443 (2019).
20. Brown, G. W. Iterative solution of games by fictitious play. *Act. Anal. Prod. Alloc.* **13**, 374–376 (1951).
21. Leslie, D. S. & Collins, E. J. Generalised weakened fictitious play. *Games Econ. Behav.* **56**, 285–298 (2006).
22. Heinrich, J., Lanctot, M. & Silver, D. Fictitious self-play in extensive-form games. *Proc. Intl Conf. Machine Learning* **32**, 805–813 (2015).
23. Jouppi, N. P. et al. In-datacenter performance analysis of a tensor processing unit. Preprint at <https://arxiv.org/abs/1704.04760v1> (2017).
24. Elo, A. E. *The Rating of Chessplayers, Past and Present* (Arco, 2017).

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

© The Author(s), under exclusive licence to Springer Nature Limited 2019

Methods

Game and interface

Game environment. StarCraft is a real-time strategy game that takes place in a science fiction universe. The franchise, from Blizzard Entertainment, comprises StarCraft: Brood War and StarCraft II. In this paper, we used StarCraft II. Since StarCraft was released in 1998, there has been a strong competitive community with tens of millions of dollars of prize money. The most common competitive setting of StarCraft II is 1v1, where each player chooses one of the three available races—Terran, Protoss, and Zerg—which all have distinct units and buildings, exhibit different mechanics, and necessitate different strategies when playing for and against. There is also a Random race, where the game selects the player’s race at random. Players begin with a small base and a few worker units, which gather resources to build additional units and buildings, scout the opponent, and research new technologies. A player is defeated if they lose all buildings.

There is no universally accepted notion of fairness in real-time human–computer matches, so our match conditions, interface, camera view, action rate limits, and delays were developed in consultation with professional StarCraft II players and Blizzard employees. AlphaStar’s play under these conditions was professional-player approved (see the Professional Player Statement, below). At each agent step, the policy receives an observation o_t and issues an action a_t (Extended Data Tables 1, 2) through the game interface. There can be several game time-steps (each 45 ms) per agent step.

Camera view. Humans play StarCraft through a screen that displays only part of the map along with a high-level view of the entire map (to avoid information overload, for example). The agent interacts with the game through a similar camera-like interface, which naturally imposes an economy of attention, so that the agent chooses which area it fully sees and interacts with. The agent can move the camera as an action.

Opponent units outside the camera have certain information hidden, and the agent can only target within the camera for certain actions (for example, building structures). AlphaStar can target locations more accurately than humans outside the camera, although less accurately within it because target locations (selected on a 256×256 grid) are treated the same inside and outside the camera. Agents can also select sets of units anywhere, which humans can do less flexibly using control groups. In practice, the agent does not seem to exploit these extra capabilities (see the Professional Player Statement, below), because of the human prior. Ablation data in Fig. 3h shows that using this camera view reduces performance.

APM limits. Humans are physically limited in the number of actions per minute (APM) they can execute. Our agent has a monitoring layer that enforces APM limitations. This introduces an action economy that requires actions to be prioritized. Agents are limited to executing at most 22 non-duplicate actions per 5-s window. Converting between actions and the APM measured by the game is non-trivial, and agent actions are hard to compare with human actions (computers can precisely execute different actions from step to step). See Fig. 2c and Extended Data Fig. 1 for APM details.

Delays. Humans are limited in how quickly they react to new information; AlphaStar has two sources of delays. First, in real-time evaluation (not training), AlphaStar has a delay of about 110 ms between when a frame is observed and when an action is executed, owing to latency, observation processing, and inference. Second, because agents decide ahead of time when to observe next (on average 370 ms, but possibly multiple seconds), they may react late to unexpected situations. The distribution of these delays is shown in Extended Data Fig. 2.

Related work

Games have been a focus of artificial intelligence research for decades as a stepping stone towards more general applications. Classic board games such as chess²⁵ and Go²⁶ have been mastered using general-purpose reinforcement learning and planning algorithms¹⁸. Reinforcement learning methods have achieved substantial successes in video games such as those on the Atari platform²⁷, Super Mario Bros²⁸, Quake III Arena Capture the Flag²⁹, and Dota 2³⁰.

Real-time strategy (RTS) games are recognized for their game-theoretic and domain complexities³¹. Many sub-problems of RTS games, for example, micromanagement, base economy, or build order optimization, have been studied in depth^{7,32–35}, often in small-scale environments^{36,37}. For the combined challenge, the StarCraft domain has emerged by consensus as a research focus^{1,7}. StarCraft: Brood War has an active competitive AI research community³⁸, and most bots combine rule-based heuristics with other AI techniques such as search^{4,39}, data-driven build-order selection⁴⁰, and simulation⁴¹. Reinforcement learning has also been studied to control units in the game^{7,34,42–44}, and imitation learning has been proposed to learn unit and building compositions⁴⁵. Most recently, deep learning has been used to predict future game states⁴⁶. StarCraft II similarly has an active bot community³ since the release of a public application programming interface (API)⁷. No StarCraft bots have defeated professional players, or even high-level casual players⁴⁷, and the most successful bots have used superhuman capabilities, such as executing tens of thousands of APM or viewing the entire map at once. These capabilities make comparisons against humans hard, and invalidate certain strategies. Some of the most recent approaches use reinforcement learning to play the full game, with hand-crafted, high-level actions⁴⁸, or rule-based systems with machine learning incrementally replacing components⁴³. By contrast, AlphaStar uses a model-free, end-to-end learning approach to playing StarCraft II that sidesteps the difficulties of search-based methods that result from imperfect models, and is applicable to any domain that shares some of the challenges present in StarCraft.

Dota 2 is a modern competitive team game that shares some complexities of RTS games such as StarCraft (including imperfect information and large time horizons). Recently, OpenAI Five defeated a team of professional Dota 2 players and 99.4% of online players³⁰. The hero units of OpenAI Five are controlled by a team of agents, trained together with a scaled up version of PPO⁴⁹, based on handcrafted rewards. However, unlike AlphaStar, some game rules were simplified, players were restricted to a subset of heroes, agents used hard-coded sub-systems for certain aspects of the game, and agents did not limit their perception to a camera view.

AlphaStar relies on imitation learning combined with reinforcement learning, which has been used several times in the past. Similarly to the training pipeline of AlphaStar, the original AlphaGo initialized a policy network by supervised learning from human games, which was then used as a prior in Monte-Carlo tree search²⁶. Similar to our statistic z , other work attempted to train reward functions from human preferences and use them to guide reinforcement learning^{50,51} or learned goals from human intervention⁵².

Related to the league, recent progress in multi-agent research has led to agents performing at human level in the Capture the Flag team mode of Quake III Arena²⁹. These results were obtained using population-based training of several agents competing with each other, which used pseudo-reward evolution to deal with the hard credit assignment problem. Similarly, the Policy Space Response Oracle framework⁵³ is related to league training, although league training specifies unique targets for approximate best responses (that is, PFSP and exploiters).

Architecture

The policy of AlphaStar is a function $\pi_\theta(a_t | s_t, z)$ that maps all previous observations and actions $s_t = o_{1:t}, a_{1:t-1}$ (defined in Extended Data Tables 1, 2)

and z (representing strategy statistics) to a probability distribution over actions a_t for the current step. π_θ is implemented as a deep neural network with the following structure.

The observations o_t are encoded into vector representations, combined, and processed by a deep LSTM⁹, which maintains memory between steps. The action arguments a_t are sampled autoregressively¹⁰, conditioned on the outputs of the LSTM and the observation encoders. There is a value function for each of the possible rewards (see Reinforcement learning).

Architecture components were chosen and tuned with respect to their performance in supervised learning, and include many recent advances in deep learning architectures^{7,8,12,54,55}. A high-level overview of the agent architecture is given in Extended Data Fig. 3, with more detailed descriptions in Supplementary Data, Detailed Architecture. AlphaStar has 139 million weights, but only 55 million weights are required during inference. Ablation Fig. 3f compares the impact of scatter connections, transformer, and pointer network.

Supervised learning

Each agent is initially trained through supervised learning on replays to imitate human actions. Supervised learning is used both to initialize the agent and to maintain diverse exploration⁵⁶. Because of this, the primary goal is to produce a diverse policy that captures StarCraft's complexities.

We use a dataset of 971,000 replays played on StarCraft II versions 4.8.2 to 4.8.6 by players with MMR scores (Blizzard's metric, similar to Elo) greater than 3,500, that is, from the top 22% of players. Instructions for downloading replays can be found at <https://github.com/Blizzard/s2client-proto>. The observations and actions are returned by the game's raw interface (Extended Data Tables 1, 2). We train one policy for each race, with the same architecture as the one used during reinforcement learning.

From each replay, we extract a statistic z that encodes each player's build order, defined as the first 20 constructed buildings and units, and cumulative statistics, defined as the units, buildings, effects, and upgrades that were present during a game. We condition the policy on z in both supervised and reinforcement learning, and in supervised learning we set z to zero 10% of the time.

To train the policy, at each step we input the current observations and output a probability distribution over each action argument (Extended Data Table 2). For these arguments, we compute the KL divergence between human actions and the policy's outputs, and apply updates using the Adam optimizer⁵⁷. We also apply L_2 regularization⁵⁸. The pseudocode of the supervised training algorithm can be found in Supplementary Data, Pseudocode.

We further fine-tune the policy using only winning replays with MMR above 6,200 (16,000 games). Fine-tuning improved the win rate against the built-in elite bot from 87% to 96% in Protoss versus Protoss games. The fine-tuned supervised agents were rated at 3,947 MMR for Terran, 3,607 MMR for Protoss and 3,544 MMR for Zerg. They are capable of building all units in the game, and are qualitatively diverse from game to game (Extended Data Fig. 4).

Reinforcement learning

We apply reinforcement learning to improve the performance of AlphaStar based on agent-versus-agent games. We use the match outcome (−1 on a loss, 0 on a draw and +1 on a win) as the terminal reward r_T , without a discount to accurately reflect the true goal of winning games. Following the actor–critic paradigm¹⁴, a value function $V_\theta(s_t, z)$ is trained to predict r_t , and used to update the policy $\pi_\theta(a_t | s_t, z)$.

StarCraft poses several challenges when viewed as a reinforcement learning problem: exploration is difficult, owing to domain complexity and reward sparsity; policies need to be capable of executing diverse strategies throughout training; and off-policy learning is difficult, owing to large time horizons and the complex action space.

Exploration and diversity. We use human data to aid in exploration and to preserve strategic diversity throughout training. First, we initialize the policy parameters to the supervised policy and continually minimize the KL divergence between the supervised and current policy^{59,60}. Second, we train the main agents with pseudo-rewards to follow a strategy statistic z , which we randomly sample from human data. These pseudo-rewards measure the edit distance between sampled and executed build orders, and the Hamming distance between sampled and executed cumulative statistics (see Supplementary Data, Detailed Architecture). Each type of pseudo-reward is active (that is, non-zero) with probability 25%, and separate value functions and losses are computed for each pseudo-reward. We found our use of human data to be critical in achieving good performance with reinforcement learning (Fig. 3e).

Value and policy updates. New trajectories are generated by actors. Asynchronously, model parameters are updated by learners, using a replay buffer that stores trajectories. Because of this, AlphaStar is subject to off-policy data, which potentially requires off-policy corrections. We found that existing off-policy correction methods^{14,61} can be inefficient in large, structured action spaces such as that used for StarCraft, because distinct actions can result in similar (or even identical) behaviour. We addressed this by using a hybrid approach that combines off-policy corrections for the policy (which avoids instability), with an uncorrected update of the value function (which introduces bias but reduces variance). Specifically, the policy is updated using V-trace and the value estimates are updated using TD(λ)⁵ (ablation in Fig. 3i). When applying V-trace to the policy in large action spaces, the off-policy corrections truncate the trace early; to mitigate this problem, we assume independence between the action type, delay, and all other arguments, and so update the components of the policy separately. To decrease the variance of the value estimates, we also use the opponent's observations as input to the value functions (ablation in Fig. 3k). Note that these are used only during training, as value functions are unnecessary during evaluation.

In addition to the V-trace policy update, we introduce an ongoing policy update (UPGO), which updates the policy parameters in the direction of

$$\rho_t (G_t^U - V_\theta(s_t, z)) \nabla_{\theta} \log \pi_\theta(a_t | s_t, z)$$

where

$$G_t^U = \begin{cases} r_t + G_{t+1}^U & \text{if } Q(s_{t+1}, a_{t+1}, z) \geq V_\theta(s_{t+1}, z) \\ r_t + V_\theta(s_{t+1}, z) & \text{otherwise} \end{cases}$$

is an ongoing return, $Q(s_t, a_t, z)$ is an action-value estimate,

$\rho_t = \min\left(\frac{\pi_\theta(a_t | s_t, z)}{\pi_{\theta'}(a_t | s_t, z)}, 1\right)$ is a clipped importance ratio, and $\pi_{\theta'}$ is the

policy that generated the trajectory in the actor. Similar to self-imitation learning¹⁷, the idea is to update the policy from partial trajectories with better-than-expected returns by bootstrapping when the behaviour policy takes a worse-than-average action (ablation in Fig. 3i). Owing to the difficulty of approximating $Q(s_t, a_t, z)$ over the large action space of StarCraft, we estimate action-values with a one-step target, $Q(s_t, a_t, z) = r_t + V_\theta(s_{t+1}, z)$.

The overall loss is a weighted sum of the policy and value function losses described above, corresponding to the win-loss reward r_t as well as pseudo-rewards based on human data, the KL divergence loss with respect to the supervised policy, and the standard entropy regularization loss¹³. We optimize the overall loss using Adam⁵⁷. The pseudocode of the reinforcement learning algorithm can be found in Supplementary Data, Pseudocode.

Multi-agent learning

League training is a multi-agent reinforcement learning algorithm that is designed both to address the cycles commonly encountered during self-play training and to integrate a diverse range of strategies. During training, we populate the league by regularly saving the parameters from our agents (that are being trained by the RL algorithm) as new players (which have fixed, frozen parameters). We also continuously re-evaluate the internal payoff estimation, giving agents up-to-date information about their performance against all players in the league (see evaluators in Extended Data Fig. 6).

Prioritized fictitious self-play. Our self-play algorithm plays games between the latest agents for all three races. This approach may chase cycles in strategy space and does not work well in isolation (Fig. 3d). FSP^{20–22} avoids cycles by playing against all previous players in the league. However, many games are wasted against players that are defeated in almost 100% of games. Consequently, we introduce PFSP. Instead of uniformly sampling opponents in the league, we use a match-making mechanism to provide a good learning signal. Given a learning agent A , we sample the frozen opponent B from a candidate set C with probability

$$\frac{f(\mathbb{P}[A \text{ beats } B])}{\sum_{C \in \mathcal{C}} f(\mathbb{P}[A \text{ beats } C])}$$

Where $f: [0, 1] \rightarrow [0, \infty)$ is some weighting function.

Choosing $f_{\text{hard}}(x) = (1 - x)^p$ makes PFSP focus on the hardest players, where $p \in \mathbb{R}_+$ controls how entropic the resulting distribution is. As $f_{\text{hard}}(1) = 0$, no games are played against opponents that the agent already beats. By focusing on the hardest players, the agent must beat everyone in the league rather than maximizing average performance, which is even more important in highly non-transitive games such as StarCraft (Extended Data Fig. 8), where the pursuit of the mean win rate might lead to policies that are easy to exploit. This scheme is used as the default weighting of PFSP. Consequently, on the theoretical side, one can view f_{hard} as a form of smooth approximation of max–min optimization, as opposed to max–avg, which is imposed by FSP. In particular, this helps with integrating information from exploits, as these are strong but rare counter strategies, and a uniform mixture would be able to just ignore them (Extended Data Fig. 5).

Only playing against the hardest opponents can waste games against much stronger opponents, so PFSP also uses an alternative curriculum, $f_{\text{var}}(x) = x(1 - x)$, where the agent preferentially plays against opponents around its own level. We use this curriculum for main exploiters and struggling main agents.

Populating the league. During training we used three agent types that differ only in the distribution of opponents they train against, when they are snapshotted to create a new player, and the probability of resetting to the supervised parameters.

Main agents are trained with a proportion of 35% SP, 50% PFSP against all past players in the league, and an additional 15% of PFSP matches against forgotten main players the agent can no longer beat and past main exploiters. If there are no forgotten players or strong exploiters, the 15% is used for self-play instead. Every 2×10^9 steps, a copy of the agent is added as a new player to the league. Main agents never reset.

League exploiters are trained using PFSP and their frozen copies are added to the league when they defeat all players in the league in more than 70% of games, or after a timeout of 2×10^9 steps. At this point there is a 25% probability that the agent is reset to the supervised parameters. The intuition is that league exploiters identify global blind spots in the league (strategies that no player in the league can beat, but that are not necessarily robust themselves).

Main exploiters play against main agents. Half of the time, and if the current probability of winning is lower than 20%, exploiters use PFSP with f_{var} weighting over players created by the main agents. This forms a curriculum that facilitates learning. Otherwise there is enough learning signal and it plays against the current main agents. These agents are added to the league whenever all three main agents are defeated in more than 70% of games, or after a timeout of 4×10^9 steps. They are then reset to the supervised parameters. Main exploiters identify weaknesses of main agents, and consequently make them more robust.

For more details refer to the Supplementary Data, Pseudocode.

Infrastructure

In order to train the league, we run a large number of StarCraft II matches in parallel and update the parameters of the agents on the basis of data from those games. To manage this, we developed a highly scalable training setup with different types of distributed workers.

For every training agent in the league, we run 16,000 concurrent StarCraft II matches and 16 actor tasks (each using a TPU v3 device with eight TPU cores²³) to perform inference. The game instances progress asynchronously on preemptible CPUs (roughly equivalent to 150 processors with 28 physical cores each), but requests for agent steps are batched together dynamically to make efficient use of the TPU. Using TPUs for batched inference provides large efficiency gains over previous work^{14,29}.

Actors send sequences of observations, actions, and rewards over the network to a central 128-core TPU learner worker, which updates the parameters of the training agent. The received data are buffered in memory and replayed twice. The learner worker performs large-batch synchronous updates. Each TPU core processes a mini-batch of four sequences, for a total batch size of 512. The learner processes about 50,000 agent steps per second. The actors update their copy of the parameters from the learner every 10 s.

We instantiate 12 separate copies of this actor–learner setup: one main agent, one main exploiter and two league exploiter agents for each StarCraft race. One central coordinator maintains an estimate of the payoff matrix, samples new matches on request, and resets main and league exploiters. Additional evaluator workers (running on the CPU) are used to supplement the payoff estimates. See Extended Data Fig. 6 for an overview of the training setup.

Evaluation

AlphaStar Battle.net evaluation. AlphaStar agents were evaluated against humans on Battle.net, Blizzard’s online matchmaking system based on MMR ratings, on StarCraft II balance patch 4.9.3. AlphaStar Final was rated at Grandmaster level, above 99.8% of human players who were active enough in the past months to be placed into a league on the European server (about 90,000 players).

AlphaStar played only opponents who opted to participate in the experiment (the majority of players opted in)⁶², used an anonymous account name, and played on four maps: Cyber Forest, Kairos Junction, King’s Cove, and New Repugnancy. Blizzard updated the map pool a few weeks before testing. Instead of retraining AlphaStar, we simply played on the four common maps that were kept in the pool of seven available maps. Humans also must select at least four maps and frequently play under anonymous account names. Each agent ran on a single high-end consumer GPU. We evaluated at three points during training: supervised, midpoint, and final.

For the supervised and midpoint evaluations, each agent began with a fresh, unranked account. Their MMR was updated on Battle.net as for humans. The supervised and midpoint evaluations played 30 and 60 games, respectively. The midpoint evaluation was halted while still increasing because the anonymity constraint was compromised after 50 games.

For the final Battle.net evaluation, we used several accounts to parallelize the games and help to avoid identification. The MMRs of our

Article

accounts were seeded randomly from the distribution of combined, estimated, midpoint MMRs. Consequently, we no longer used the iterative MMR estimation provided in Battle.net, and instead used the underlying probabilistic model provided by Blizzard: given our rating r with uncertainty u , and opponent rating r_i with uncertainty $u_i \in [0.1, 1.0]$, the probability of the outcome $o_i \in \{-1, 1\}$ is

$$\begin{aligned}\mathbb{P}[o_i = 1 \mid r, u, r_i, u_i] &= 1 - \mathbb{P}[o_i = -1 \mid r, u, r_i, u_i] \\ &= \Phi\left(\frac{r - r_i}{400\sqrt{2 + u^2 + u_i^2}}\right) \\ &\approx \Phi\left(\frac{r - r_i}{568}\right)\end{aligned}$$

where Φ is the cumulative distribution function (CDF) of a standard Gaussian distribution, and where we used Battle.net's minimum uncertainties $u = u_i = 0.1$.

Under independent and identically distributed (IID) assumptions of match results and a uniform prior over MMRs, we can compute our rating as

$$\begin{aligned}\operatorname{argmax}_{r \in \mathbb{N}} \mathbb{P}[r \mid \text{results}] &= \operatorname{argmax}_{r \in \mathbb{N}} \mathbb{P}[\text{results} \mid r] U(r) \\ &= \operatorname{argmax}_{r \in \mathbb{N}} \prod_{i=1}^N \mathbb{P}[o_i \mid r, r_i]\end{aligned}$$

We validated our MMR computation on the 200 most recent matches of Dario 'TLO' Wunsch, a professional StarCraft II player, and obtained an MMR estimate of 6,334; the average MMR reported by Battle.net was 6,336.

StarCraft demonstration evaluation. In December 2018, we played two five-game series against StarCraft II professional players Grzegorz 'MaNa' Komincz and Dario 'TLO' Wunsch, although TLO did not play the same StarCraft II race that he plays professionally. These games took place with a different, preliminary version of AlphaStar⁶³. In particular, the agent did not have a limited camera, was less restricted in how often it could act, and played for and against a single StarCraft II race on a single map. AlphaStar won all ten games in both five-game series, although an early camera prototype lost a follow-up game against MaNa.

Analysis

Agent sets. For validation agents, we validated league robustness against a set of 17 strategies trained using only main agents and no exploiters, and fixing z to a hand-curated set of interesting strategies (for example, a cannon rush or early flying units).

Ablation test agents included the validation agents, and the first (that is, weaker) 20 main and 20 league exploiter Protoss agents created by full league training.

For fixed opponents, to evaluate our reinforcement learning algorithms, we computed the best response against a uniform mixed strategy composed of the first ten league exploiter Protoss agents created by league training.

Metrics used in Figures. To compute internal Elo ratings of the league, we added the built-in bots, and used them to estimate Elo with the following model:

$$\mathbb{P}[r_1 \text{ beats } r_2] = \frac{1}{1 + e^{-(r_1 - r_2)/400}} \approx \Phi\left(\frac{r_1 - r_2}{400}\right)$$

where r_1 and r_2 are the Elo ratings of both players. As the Elo rating has no intrinsic absolute scale, we ground it by setting the rating of the built-in elite bot to 0.

RPP is the expected outcome of the meta-game between two populations after they reach the Nash equilibrium¹⁹. Given a payoff matrix

between all agents in leagues A and B of sizes N and M , respectively, $P_{AB} \in [0, 1]^{N \times M}$:

$$\text{RPP}(P_{AB}) = \text{Nash}(P_{AB})^T P_{AB} \text{Nash}(P_{BA})$$

where $\text{Nash}(X) \in [0, 1]^K$ is a vector of probabilities assigned to playing each agent, in league X of size K , in the Nash equilibrium. High RPP means that league A consists of agents that can form a mixed strategy that can exploit agents from league B , while not being too exploitable by any mixed strategy from league B .

AlphaStar generality

To address the complexity and game-theoretic challenges of StarCraft, AlphaStar uses a combination of new and existing general-purpose techniques for neural network architectures, imitation learning, reinforcement learning, and multi-agent learning. These techniques and their combination are widely applicable.

The neural network architecture components, including the new scatter connections, are all generally applicable to any domain whose observations comprise a combination of images, lists, and sets, all of which are present in StarCraft.

AlphaStar's action space is defined as a set of functions with typed arguments. Any domain which defines a similar API can be tackled with the same decomposition of complex, structured action spaces, whose joint probability is decomposed via the chain rule (akin to, for example, language modelling¹⁰ or theorem proving).

Imitation learning for AlphaStar requires a large number of human demonstrations to be effective, and thus is applicable only to those domains that provide such a set of demonstrations. Using a latent variable z to induce exploration is not specific to StarCraft, but the particular choice of statistics required domain knowledge. In particular, we chose z to encode openings and units in StarCraft. Pseudo-rewards were based on appropriate distance metrics for these statistics, such as edit distance or Hamming distance.

AlphaStar's underlying reinforcement learning algorithm can be applied to any reinforcement learning environment. The use of an opponent's observations for a lower-variance baseline and new components, such as hybrid off-policy learning, UPGO, and distillation towards an imitation policy, are also widely applicable.

Last, we propose a new multi-agent training regime with different kinds of exploiters whose purpose is to strengthen the main agents. Together with PFSP, these are all general-purpose techniques that can be applied to any multiplayer domain.

Professional player statement

The following quote describes our interface and limitations from StarCraft II professional player Dario 'TLO' Wunsch (who is part of the team and an author of this paper).

"The limitations that have been put in place for AlphaStar now mean that it feels very different from the initial show match in January. While AlphaStar has excellent and precise control it doesn't feel superhuman—certainly not on a level that a human couldn't theoretically achieve. It is better in some aspects than humans and then also worse in others, but of course there are going to be unavoidable differences between AlphaStar and human players.

I've had the pleasure of providing consultation to the AlphaStar team to help ensure that DeepMind's system does not have any unfair advantages over human players. Overall, it feels very fair, like it is playing a 'real' game of StarCraft and doesn't completely throw the balance off by having unrealistic capabilities. Now that it has limited camera view, when I multi-task it doesn't always catch everything at the same time, so that aspect also feels very fair and more human-like."

Reporting summary

Further information on research design is available in the Nature Research Reporting Summary linked to this paper.

Data availability

All the games that AlphaStar played online can be found in the file ‘replays.zip’ in the Supplementary Data, and the raw data from the Battle.net experiment can be found in ‘bnet.json’ in the Supplementary Data.

Code availability

The StarCraft II environment was open sourced in 2017 by Blizzard and DeepMind⁷. All the human replays used for imitation learning can be found at <https://github.com/Blizzard/s2client-proto>. The pseudocode for the supervised learning, reinforcement learning, and multi-agent learning components of AlphaStar can be found in the file ‘pseudocode.zip’ in the Supplementary Data. All the neural architecture details and hyper-parameters can be found in the file ‘detailed-architecture.txt’ in the Supplementary Data.

25. Campbell, M., Hoane, A. & Hsu, F. Deep Blue. *Artif. Intell.* **134**, 57–83 (2002).
26. Silver, D. et al. Mastering the game of Go with deep neural networks and tree search. *Nature* **529**, 484–489 (2016).
27. Mnih, V. et al. Human-level control through deep reinforcement learning. *Nature* **518**, 529–533 (2015).
28. Pathak, D., Agrawal, P., Efros, A. A. & Darrell, T. Curiosity-driven exploration by self-supervised prediction. *Proc. IEEE Conf. Computer Vision Pattern Recognition Workshops* 16–17 (IEEE, 2017).
29. Jaderberg, M. et al. Human-level performance in 3D multiplayer games with population-based reinforcement learning. *Science* **364**, 859–865 (2019).
30. OpenAI OpenAI Five. <https://blog.openai.com/openai-five/> (2018).
31. Buro, M. Real-time strategy games: a new AI research challenge. *Intl Joint Conf. Artificial Intelligence* 1534–1535 (2003).
32. Samvelyan, M. et al. The StarCraft multi-agent challenge. *Intl Conf. Autonomous Agents and MultiAgent Systems* 2186–2188 (2019).
33. Zambaldi, V. et al. Relational deep reinforcement learning. Preprint at <https://arxiv.org/abs/1806.01830v2> (2018).
34. Usunier, N., Synnaeve, G., Lin, Z. & Chintala, S. Episodic exploration for deep deterministic policies: an application to StarCraft micromanagement tasks. Preprint at <https://arxiv.org/abs/1609.02993v3> (2017).
35. Weber, B. G. & Mateas, M. Case-based reasoning for build order in real-time strategy games. *AIIDE '09 Proc. 5th AAAI Conf. Artificial Intelligence and Interactive Digital Entertainment* 106–111 (2009).
36. Buro, M. ORTS: a hack-free RTS game environment. *Intl Conf. Computers and Games* 280–291 (Springer, 2002).
37. Churchill, D. SparCraft: open source StarCraft combat simulation. <https://code.google.com/archive/p/sparcraft/> (2013).
38. Weber, B. G. AIIDE 2010 StarCraft competition. *Artificial Intelligence and Interactive Digital Entertainment Conf.* (2010).
39. Uriarte, A. & Ontañón, S. Improving Monte Carlo tree search policies in StarCraft via probabilistic models learned from replay data. *Artificial Intelligence and Interactive Digital Entertainment Conf.* 101–106 (2016).
40. Hsieh, J.-L. & Sun, C.-T. Building a player strategy model by analyzing replays of real-time strategy games. *IEEE Intl Joint Conf. Neural Networks* 3106–3111 (2008).
41. Synnaeve, G. & Bessiere, P. A Bayesian model for plan recognition in RTS games applied to StarCraft. *Artificial Intelligence and Interactive Digital Entertainment Conf.* 79–84 (2011).
42. Shao, K., Zhu, Y. & Zhao, D. StarCraft micromanagement with reinforcement learning and curriculum transfer learning. *IEEE Trans. Emerg. Top. Comput. Intell.* **3**, 73–84 (2019).
43. Facebook CherryPi. <https://torchcraft.github.io/TorchCraftAI/>.
44. Berkeley Overmind. <https://www.icsi.berkeley.edu/icsi/news/2010/10/klein-berkeley-overmind> (2010).
45. Justesen, N. & Risi, S. Learning macromanagement in StarCraft from replays using deep learning. *IEEE Conf. Computational Intelligence and Games (CIG)* 162–169 (2017).

46. Synnaeve, G. et al. Forward modeling for partial observation strategy games—a StarCraft defogger. *Adv. Neural Information Process. Syst.* **31**, 10738–10748 (2018).
47. Farooq, S. S., Oh, I.-S., Kim, M.-J. & Kim, K. J. StarCraft AI competition report. *AI Mag.* **37**, 102–107 (2016).
48. Sun, P. et al. TStarBots: defeating the cheating level builtin AI in StarCraft II in the full game. Preprint at <https://arxiv.org/abs/1809.07193v3> (2018).
49. Schulman, J., Wolski, F., Dhariwal, P., Radford, A. & Klimov, O. Proximal policy optimization algorithms. Preprint at <https://arxiv.org/abs/1707.06347v2> (2017).
50. Ibarz, B. et al. Reward learning from human preferences and demonstrations in Atari. *Adv. Neural Information Process. Syst.* **31**, 8011–8023 (2018).
51. Nair, A., McGrew, B., Andrychowicz, M., Zaremba, W. & Abbeel, P. Overcoming exploration in reinforcement learning with demonstrations. *IEEE Intl Conf. Robotics and Automation* 6292–6299 (2018).
52. Christiano, P. F. et al. Deep reinforcement learning from human preferences. *Adv. Neural Information Process. Syst.* **30**, 4299–4307 (2017).
53. Lanctot, M. et al. A unified game-theoretic approach to multiagent reinforcement learning. *Adv. Neural Information Process. Syst.* **30**, 4190–4203 (2017).
54. Perez, E., Strub, F., De Vries, H., Dumoulin, V. & Courville, A. FiLM: visual reasoning with a general conditioning layer. Preprint at <https://arxiv.org/abs/1709.07871v2> (2018).
55. He, K., Zhang, X., Ren, S. & Sun, J. Deep residual learning for image recognition. *Proc. IEEE Conf. Computer Vision and Pattern Recognition* 770–778 (2016).
56. Hinton, G., Vinyals, O. & Dean, J. Distilling the knowledge in a neural network. Preprint at <https://arxiv.org/abs/1503.02531v1> (2015).
57. Kingma, D. P. & Ba, J. Adam: a method for stochastic optimization. Preprint at <https://arxiv.org/abs/1412.6980v9> (2014).
58. Bishop, C. M. *Pattern Recognition and Machine Learning* (Springer, 2006).
59. Rusu, A. A. et al. Policy distillation. Preprint at <https://arxiv.org/abs/1511.06295> (2016).
60. Parisotto, E., Ba, J. & Salakhutdinov, R. Actor-mimic: deep multitask and transfer reinforcement learning. Preprint at <https://arxiv.org/abs/1511.06342> (2016).
61. Precup, D., Sutton, R. S. & Singh, S. P. Eligibility traces for off-policy policy evaluation. *ICML '00 Proc. 17th Intl Conf. Machine Learning* 759–766 (2016).
62. DeepMind Research on Ladder. <https://starcraft2.com/en-us/news/22933138> (2019).
63. Vinyals, O. et al. AlphaStar: mastering the real-time strategy game StarCraft II <https://deepmind.com/blog/article/alphastar-mastering-real-time-strategy-game-starcraft-ii> (DeepMind, 2019).

Acknowledgements We thank Blizzard for creating StarCraft and for their continued support of the research environment, and for enabling AlphaStar to participate in Battle.net. In particular, we thank A. Hudelson, C. Lee, K. Calderone, and T. Morten. We also thank StarCraft II professional players G. ‘MaNa’ Komincz and D. ‘Kelazhur’ Schwimer for their StarCraft expertise and advice. We thank A. Cain, A. Razavi, D. Toyama, D. Balduzzi, D. Fritz, E. Aygün, F. Strub, G. Ostrovski, G. Alain, H. Tang, J. Sanchez, J. Fildes, J. Schrittwieser, J. Novosad, K. Simonyan, K. Kurach, P. Hamel, R. Barreira, S. Reed, S. Bartunov, S. Mourad, S. Gaffney, T. Hubert, the team that created PySC2 and the whole DeepMind Team, with special thanks to the research platform team, comms and events teams, for their support, ideas, and encouragement.

Author contributions O.V., I.B., W.M.C., M.M., A.D., J.C., D.H.C., R.P., T.E., P.G., J.O., D. Horgan, M.K., I.D., A.H., L.S., T.C., J.P.A., C.A., and D.S. contributed equally. O.V., I.B., W.M.C., M.M., A.D., J.C., D.H.C., R.P., T.E., P.G., J.O., D. Horgan, M.K., I.D., A.H., L.S., T.C., J.P.A., C.A., R.L., M.J., V.D., Y.S., A.S.V., D.B., T.L.P., C.G., Z.W., T. Pfaff, T. Pohlen, Y.W., and D.S. designed and built AlphaStar with advice from T.S. and T.L. J.M. and R.R. contributed to software engineering. D.W. and D.Y. provided expertise in the StarCraft II domain. K.K., D. Hassabis, K.M., O.S., and C.A. managed the project. D.S., W.M.C., O.V., J.O., I.B., and D.H.C. wrote the paper with contributions from M.M., J.C., D. Horgan, L.S., R.L., T.C., T.S., and T.L. O.V. and D.S. led the team.

Competing interests M.J., W.M.C., O.V., and D.S. have filed provisional patent application 62/796,567 about the contents of this manuscript. The remaining authors declare no competing interests.

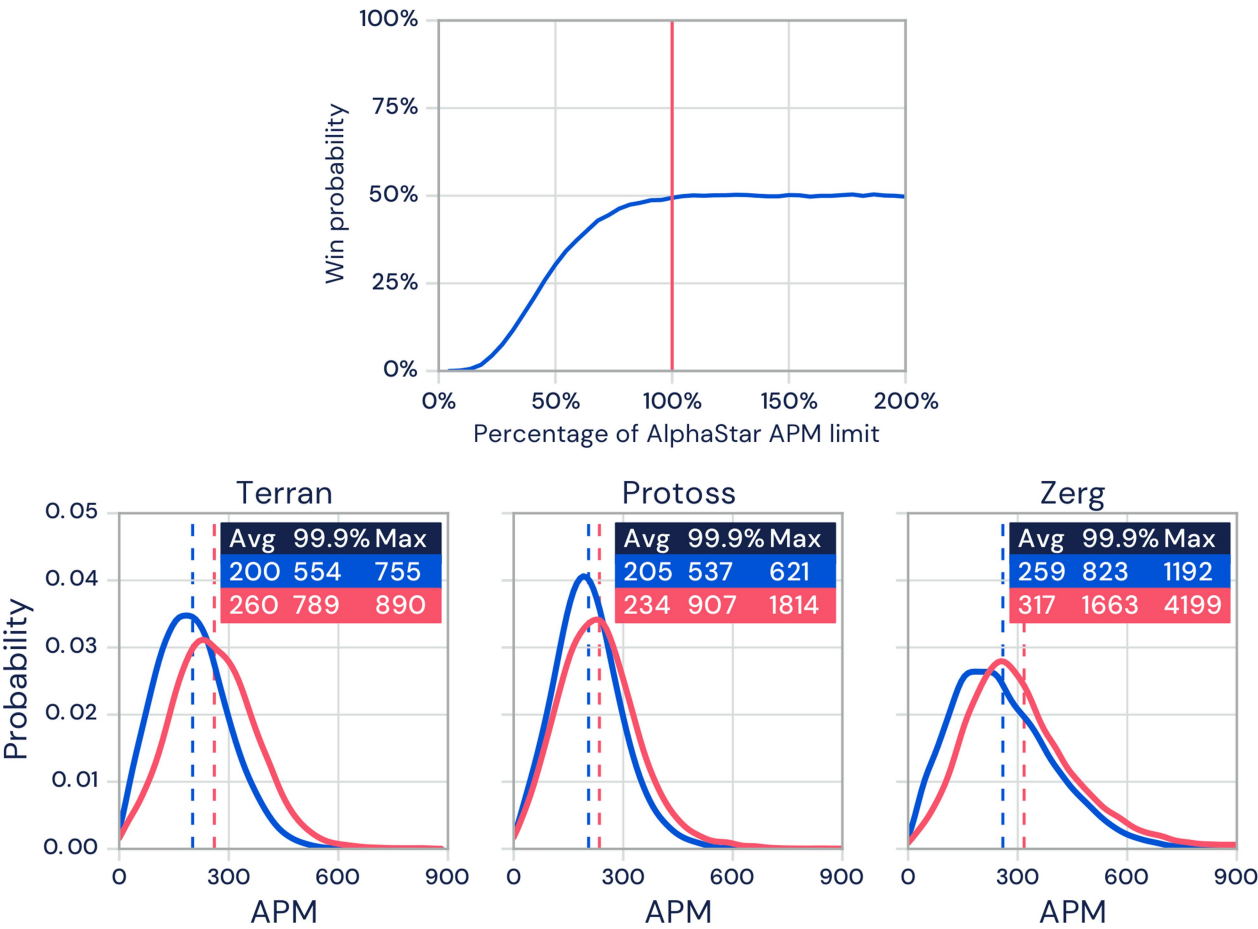
Additional information

Supplementary information is available for this paper at <https://doi.org/10.1038/s41586-019-1724-z>.

Correspondence and requests for materials should be addressed to O.V. or D.S.

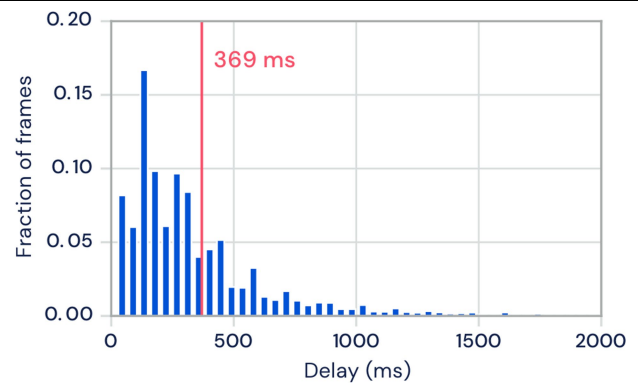
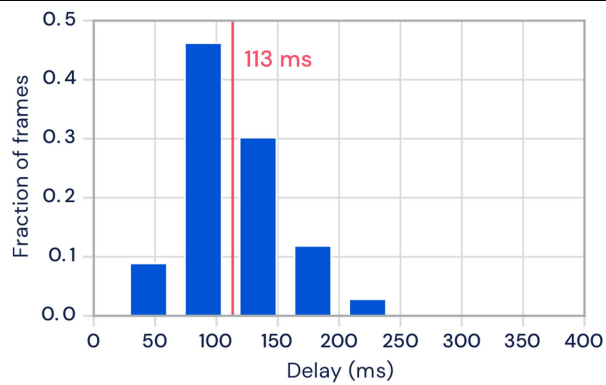
Peer review information Nature thanks Dave Churchill, Santiago Ontanon and the other, anonymous, reviewer(s) for their contribution to the peer review of this work.

Reprints and permissions information is available at <http://www.nature.com/reprints>.

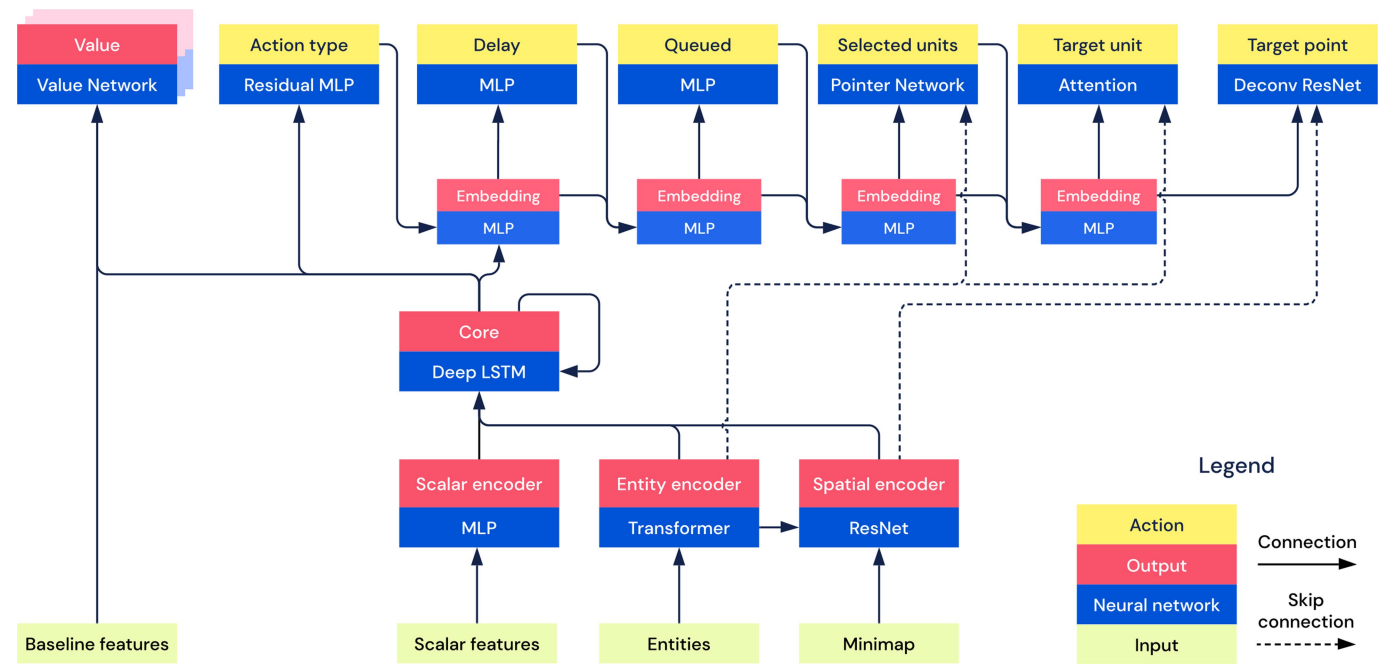


Extended Data Fig. 1 | APM limits. Top, win probability of AlphaStar Supervised against itself, when applying various agent action rate limits. Our limit does not affect supervised performance and is acceptable when

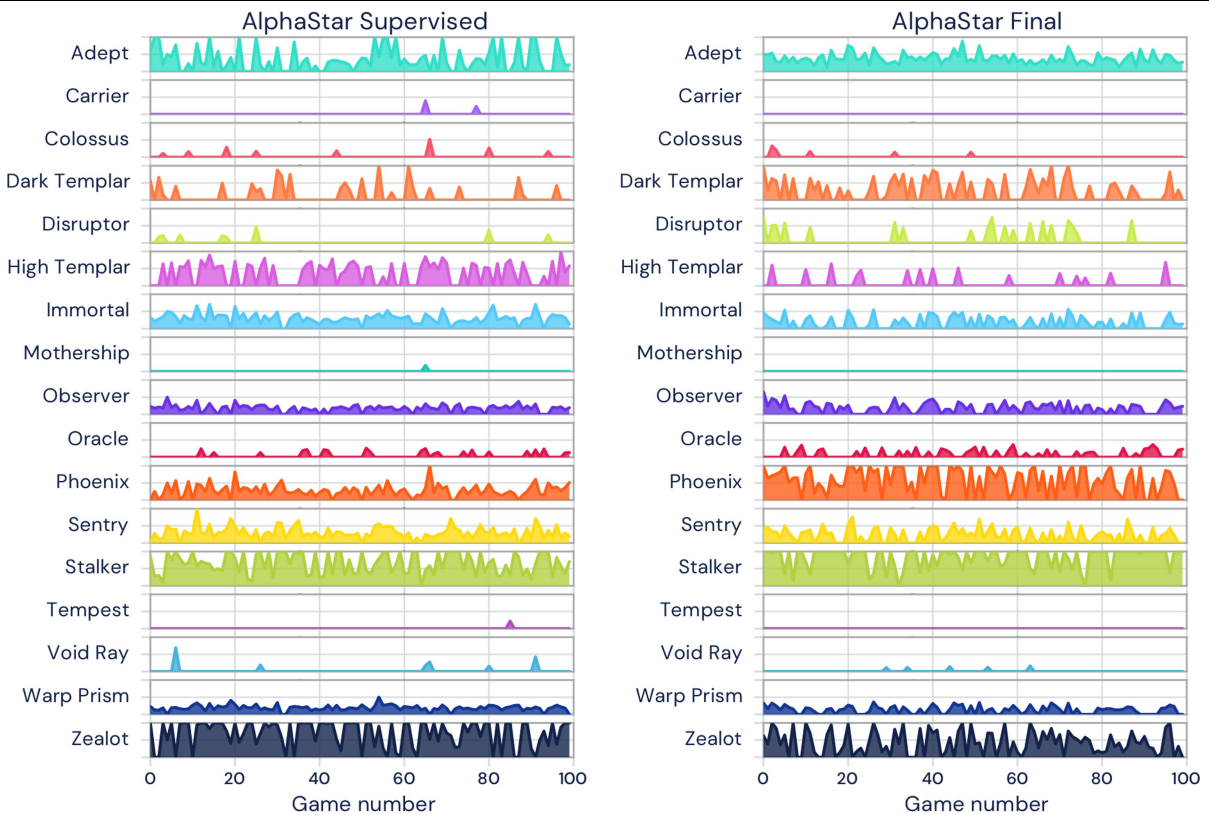
compared to humans. Bottom, distributions of APMs of AlphaStar Final (blue) and humans (red) during games on Battle.net. Dashed lines show mean values.



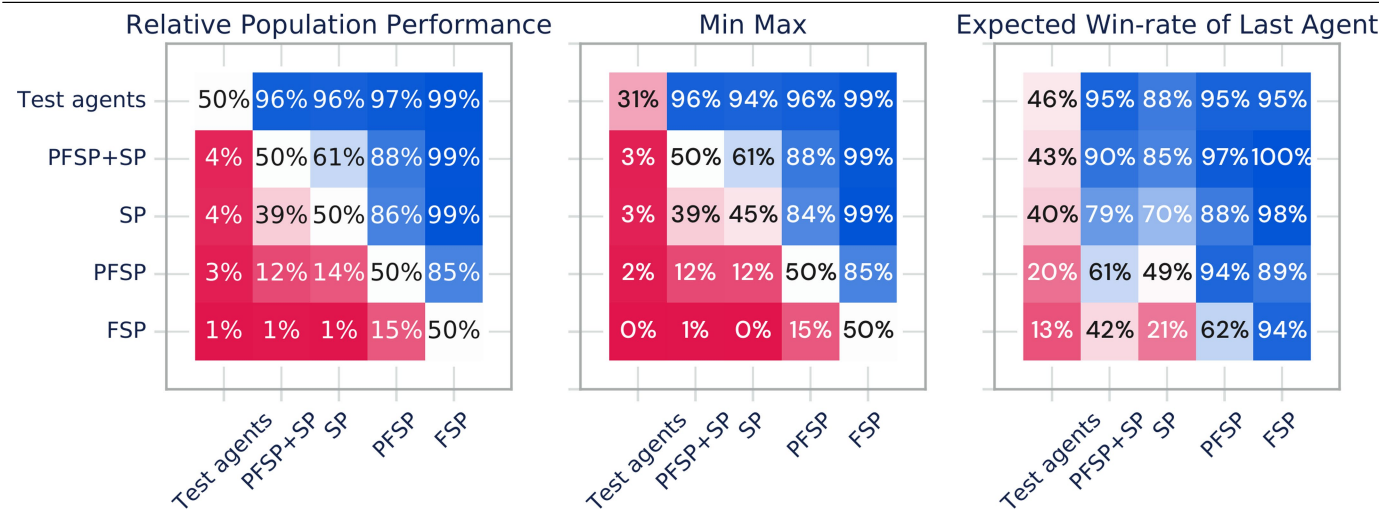
Extended Data Fig. 2 | Delays. Left, distribution of delays between when the game generates an observation and when the game executes the corresponding agent action. Right, distribution of how long agents request to wait without observing between observations.



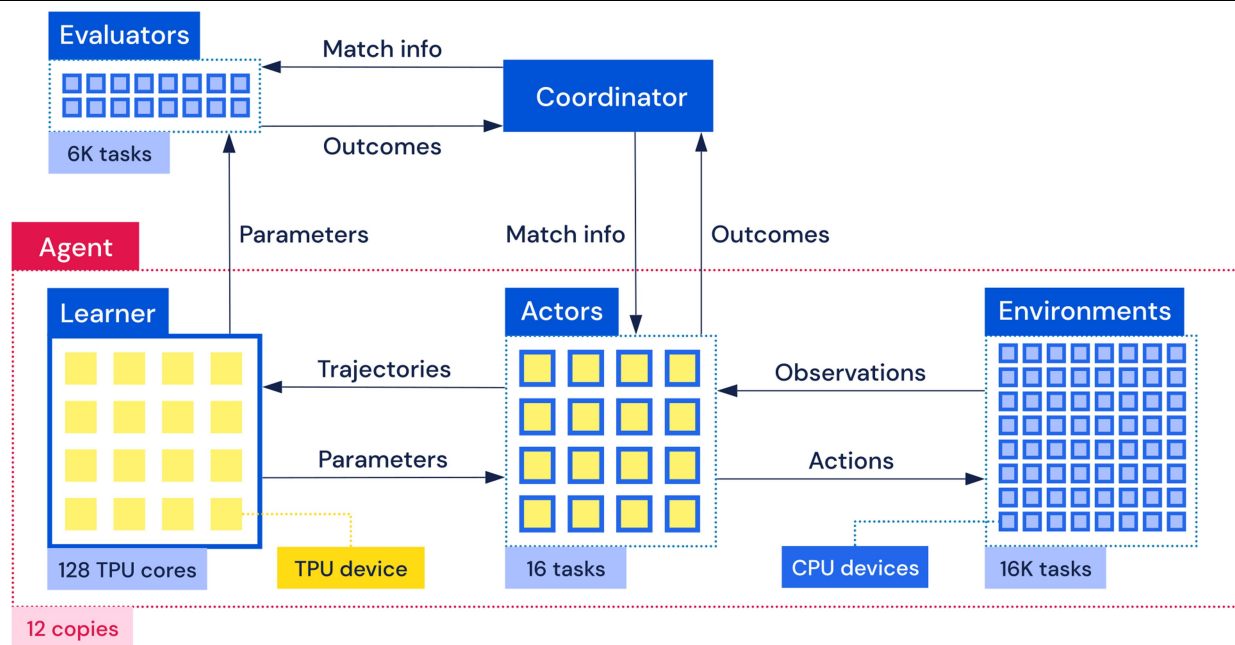
Extended Data Fig. 3 | Overview of the architecture of AlphaStar. A detailed description is provided in the Supplementary Data, Detailed Architecture.



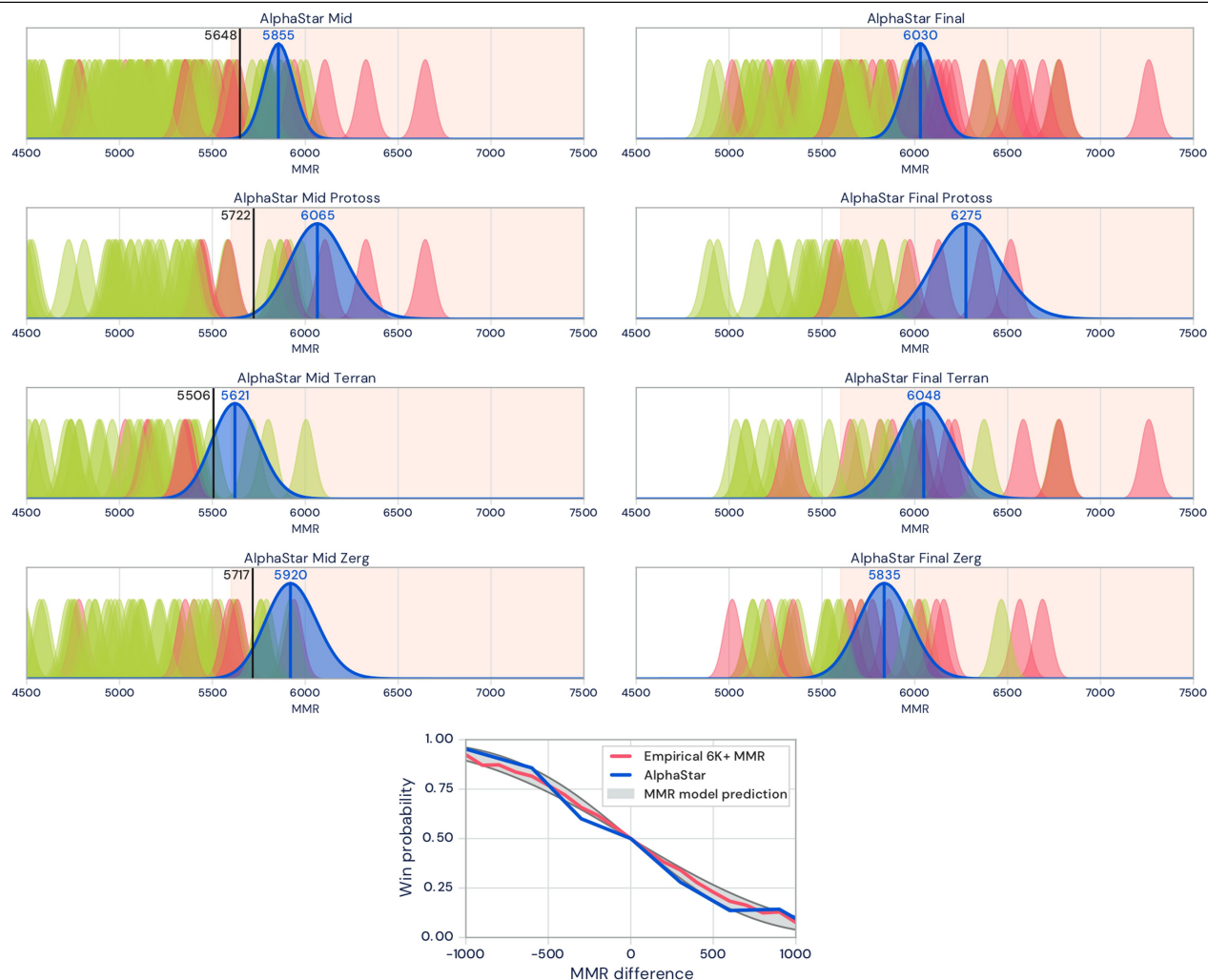
Extended Data Fig. 4 | Distribution of units built in a game. Units built by Protoss AlphaStar Supervised (left) and AlphaStar Final (right) over multiple self-play games. AlphaStar Supervised can build every unit.



Extended Data Fig. 5 | A more detailed analysis of multi-agent ablations from Fig. 3c, d. PFSP-based training outperforms FSP under all measures considered: it has a stronger population measured by relative population performance, provides a less exploitable solution, and has better final agent performance against the corresponding league.

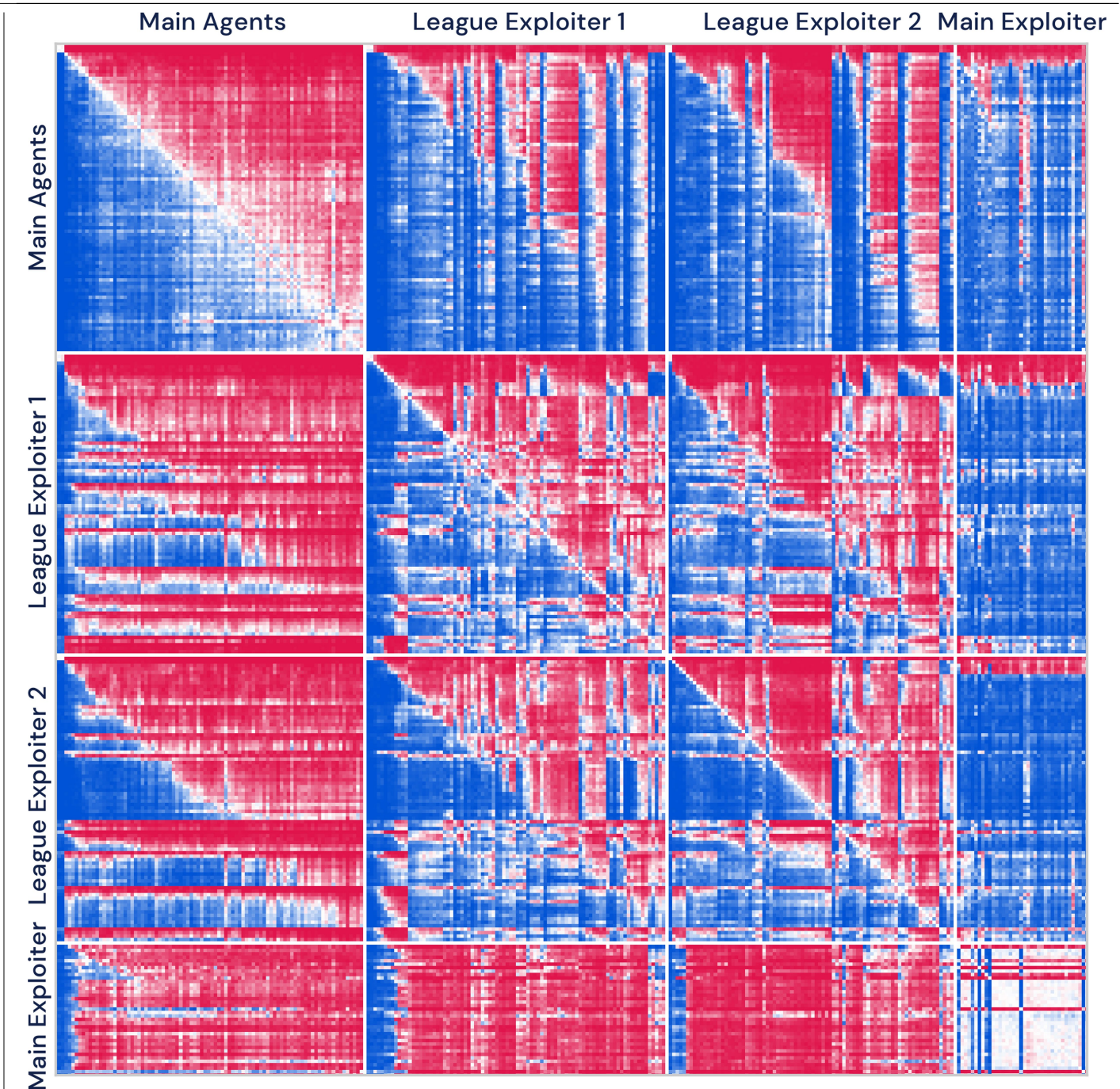


Extended Data Fig. 6 | Training infrastructure. Diagram of the training setup for the entire league.



Extended Data Fig. 7 | Battle.net performance details. Top, visualization of all the matches played by AlphaStar Final (right) and matches against opponents above 4,500 MMR of AlphaStar Mid (left). Each Gaussian represents an opponent MMR (with uncertainty): AlphaStar won against opponents shown in green and lost to those shown in red. Blue is our MMR estimate, and black is the MMR reported by StarCraft II. The orange background is the Grandmaster

league range. Bottom, win probability versus gap in MMR. The shaded grey region shows MMR model predictions when players' uncertainty is varied. The red and blue line are empirical win rates for players above 6,000 MMR and AlphaStar Final, respectively. Both human and AlphaStar win rates closely follow the MMR model.



Extended Data Fig. 8 | Payoff matrix (limited to only Protoss versus Protoss games for simplicity) split into agent types of the league. Blue means a row agent wins, red loses, and white draws. The main agents behave transitively: the more recent agents win consistently against older main agents

and exploiters. Interactions between exploiters are highly non-transitive: across the full payoff, there are around 3,000,000 rock-paper-scissor cycles (with requirement of at least 70% win rates to form a cycle) that involve at least one exploiter, and around 200 that involve only main agents.

Article

Extended Data Table 1 | Agent input space

Category	Field	Description
Entities: up to 512	Unit type	E.g. Drone or Forcefield
	Owner	Agent, opponent, or neutral
	Status	Current health, shields, energy
	Display type	E.g. Snapshot, for opponent buildings in the fog of war
	Position	Entity position
	Number of workers	For resource collecting base buildings
	Cooldowns	Attack cooldown
	Attributes	Invisible, powered, hallucination, active, in cargo, and/or on the screen
	Unit attributes	E.g. Biological or Armored
	Cargo status	Current and maximum amount of cargo space
	Building status	Build progress, build queue, and add-on type
	Resource status	Remaining resource contents
	Order status	Order queue and order progress
	Buff status	Bufs and buff durations
Map: 128x128 grid	Height	Heights of map locations
	Visibility	Whether map locations are currently visible
	Creep	Whether there is creep at a specific location
	Entity owners	Which player owns entities
	Alerts	Whether units are under attack
	Pathable	Which areas can be navigated over
Player data	Buildable	Which areas can be built on
	Race	Agent and opponent requested race, and agent actual race
	Upgrades	Agent upgrades and opponent upgrades, if they would be known to humans
Game statistics	Agent statistics	Agent current resources, supply, army supply, worker supply, maximum supply, number of idle workers, number of Warp Gates, and number of Larva
	Camera	Current camera position. The camera is a 32x20 game-unit sized rectangle
	Time	Current time in game

The observations received by the agent through the raw interface. Information is hidden if it would be hidden from a human player. For example, AlphaStar will not see most information about invisible opponent units unless there is a detector; opponent units hidden by the fog of war will not appear in the list of units; opponent units outside the agent’s camera view will have only the owner, display type, and position; and opponent’s cloaked units will appear in the list only if they are within the agent’s camera view. Note that this interface displays information that must be inferred or remembered by humans, such as the armour upgrades of a visible opponent unit, attack cool-downs, or entities that are occluded by other entities.

Extended Data Table 2 | Agent action space

Field	Description
Action type	Which action to execute. Some examples of actions are moving a unit, training a unit from a building, moving the camera, or no-op. See PySC2 for a full list ⁷
Selected units	Entities that will execute the action
Target	An entity or location in the map discretised to 256x256 targeted by the action
Queued	Whether to queue this action or execute it immediately
Repeat	Whether or not to issue this action multiple times
Delay	The number of game time-steps to wait until receiving the next observation

The action arguments that agents can submit through the raw interface as part of an action. Some fields may be ignored, depending on the action type.

Reporting Summary

Nature Research wishes to improve the reproducibility of the work that we publish. This form provides structure for consistency and transparency in reporting. For further information on Nature Research policies, see [Authors & Referees](#) and the [Editorial Policy Checklist](#).

Statistics

For all statistical analyses, confirm that the following items are present in the figure legend, table legend, main text, or Methods section.

n/a Confirmed

- ☐ ☒ The exact sample size (n) for each experimental group/condition, given as a discrete number and unit of measurement
- ☐ ☒ A statement on whether measurements were taken from distinct samples or whether the same sample was measured repeatedly
- ☐ ☒ The statistical test(s) used AND whether they are one- or two-sided
Only common tests should be described solely by name; describe more complex techniques in the Methods section.
- ☐ ☒ A description of all covariates tested
- ☐ ☒ A description of any assumptions or corrections, such as tests of normality and adjustment for multiple comparisons
- ☐ ☒ A full description of the statistical parameters including central tendency (e.g. means) or other basic estimates (e.g. regression coefficient) AND variation (e.g. standard deviation) or associated estimates of uncertainty (e.g. confidence intervals)
- ☒ ☐ For null hypothesis testing, the test statistic (e.g. F , t , r) with confidence intervals, effect sizes, degrees of freedom and P value noted
Give P values as exact values whenever suitable.
- ☒ ☐ For Bayesian analysis, information on the choice of priors and Markov chain Monte Carlo settings
- ☒ ☐ For hierarchical and complex designs, identification of the appropriate level for tests and full reporting of outcomes
- ☐ ☒ Estimates of effect sizes (e.g. Cohen's d , Pearson's r), indicating how they were calculated

Our web collection on [statistics for biologists](#) contains articles on many of the points above.

Software and code

Policy information about [availability of computer code](#)

Data collection

Data was collected using the publicly available version of StarCraft II (versions 4.8.2 to 4.10), developed by Blizzard Entertainment.

Data analysis

We used the open source environment to interact with the game of StarCraft II, provided by Blizzard and DeepMind (<https://github.com/deepmind/pysc2>), using the game version 4.10. The networks used the TensorFlow 1.0 library with custom extensions. Analysis was performed with custom code written in Python 2.7. We additionally provide pseudocode for all algorithms described in the paper.

For manuscripts utilizing custom algorithms or software that are central to the research but not yet described in published literature, software must be made available to editors/reviewers. We strongly encourage code deposition in a community repository (e.g. GitHub). See the Nature Research [guidelines for submitting code & software](#) for further information.

Data

Policy information about [availability of data](#)

All manuscripts must include a [data availability statement](#). This statement should provide the following information, where applicable:

- Accession codes, unique identifiers, or web links for publicly available datasets
- A list of figures that have associated raw data
- A description of any restrictions on data availability

We did provide both the raw data used in the paper from the online experiment, and all the evaluation games played in the StarCraft II standard Replay format. The dataset containing all the replays used for imitation learning are distributed by Blizzard using a specific API: <https://github.com/Blizzard/s2client-proto>

Field-specific reporting

Please select the one below that is the best fit for your research. If you are not sure, read the appropriate sections before making your selection.

☒ Life sciences ☐ Behavioural & social sciences ☐ Ecological, evolutionary & environmental sciences

For a reference copy of the document with all sections, see [nature.com/documents/nr-reporting-summary-flat.pdf](https://www.nature.com/documents/nr-reporting-summary-flat.pdf)

Life sciences study design

All studies must disclose on these points even when the disclosure is negative.

Sample size	To study our agents performance, we played a total of 360 games online against the population of players that play StarCraft II in the European servers. The sample size was determined with consultation with Blizzard and professional players, who deemed that 60 games would be sufficient to estimate performance of a new professional level player reliably with low uncertainty (less than 50 MMR). We did play 90 games total, per race, plus 30, per race, for supervised agents. For the league analysis we used around 130,000,000 full games of agent vs agent, and for ablations, we used around 20,000,000 games.
Data exclusions	No data was excluded from the study.
Replication	Because of the nature of the game, we did perform three independent experiments, using three distinct races. From the total of 9 runs, we did not observe any significant deviation, and thus we reproduced the intended conditions of the experiment ourselves. Because we played anonymously, reproducing the same conditions in future studies should be relatively easy, assuming care is taken to remain anonymous.
Randomization	The players and order in which we played against them was determined by the matchmaking algorithm that Blizzard employs to create matches in their online service, which was designed many years before our study, and which the authors of this manuscript had no control over. Such system is solely based on the skill level of players, and is thus random and the authors of this manuscript were blind to group allocation.
Blinding	The authors were blind to group allocation. See "Randomization".

Reporting for specific materials, systems and methods

We require information from authors about some types of materials, experimental systems and methods used in many studies. Here, indicate whether each material, system or method listed is relevant to your study. If you are not sure if a list item applies to your research, read the appropriate section before selecting a response.

Materials & experimental systems

n/a	Involved in the study
☒	☐ Antibodies
☒	☐ Eukaryotic cell lines
☒	☐ Palaeontology
☒	☐ Animals and other organisms
☒	☐ Human research participants
☒	☐ Clinical data

Methods

n/a	Involved in the study
☒	☐ ChIP-seq
☒	☐ Flow cytometry
☒	☐ MRI-based neuroimaging